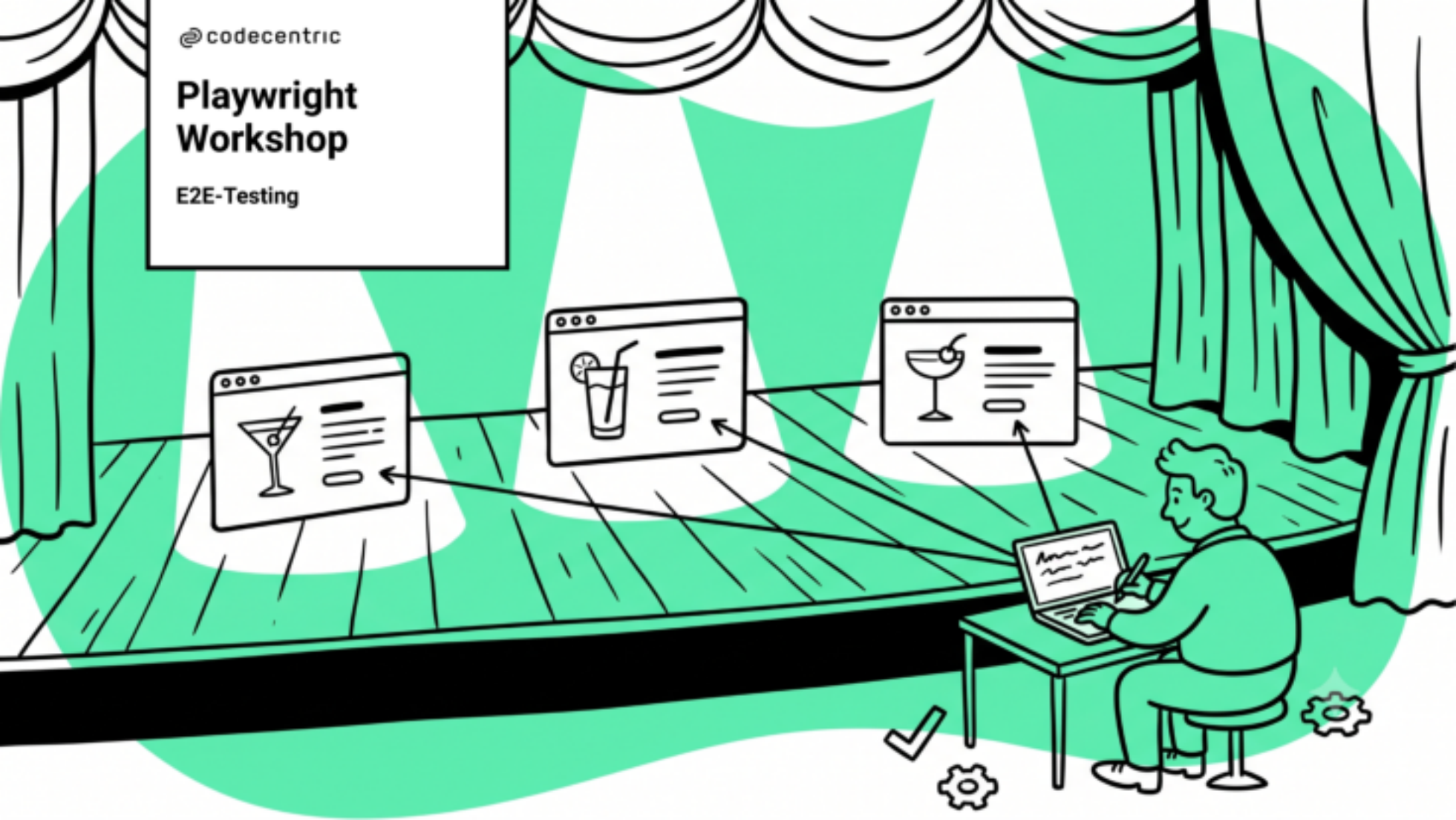


@codecentric

Playwright Workshop

E2E-Testing





1 · Einführung

Was ist Playwright?

- **E2E-Testframework** von Microsoft – testet echte Browser (Chromium, Firefox, WebKit)
- **Ein API** für alle Browser, keine separaten Driver-Binaries
- Spricht das **DevTools-Protokoll** direkt → schnell & stabil
- **Auto-Wait** eingebaut – kein manuelles Warten mehr
- Tooling im Kern: **Codegen, Trace Viewer, UI-Mode, Report** – statt drumherum gebaut
- Empfohlen: **TypeScript** (Java ist nicht Playwright-nativ)

Playwright vs. Selenium

„Selenium steuert einen Browser. Playwright testet eine Web-App.“

StateOfJS zeigt den Trend klar Richtung Playwright — Selenium bleibt stark bei Sprach-Bindungen (Java) & W3C-Standard.

Auto-Wait

Prüft vor jeder Aktion die *Actionability*. Kein

`Thread.sleep`, kein

`WebDriverWait`.

Locators statt XPath

Lazy & auto-retrying.

`getByRole`, `getByLabel`

statt brüchiger XPath.

Isolation & Tooling

Browser Contexts in ms.
Codegen, Trace, Report
eingebaut.

Zufriedenheit im Zeitverlauf

Quelle: State of JS 2025 - Testing — Ansicht auf vier Tools reduziert



Unsere Demo-App: „Happy Hour“

Eine kleine Cocktail-Bar — an ihr üben wir **alles**.

- **Menü & Suche** — Cocktails filtern, konfigurieren, in den Warenkorb legen
- **Bestellung** — Name, Tischnummer, Notiz, Trinkgeld
- **Happy-Hour-Rabatt** — zeitabhängig (hängt an Uhrzeit/Zeitzone)
- **Gutschein-Feld** — der *einzige* Netzwerk-Call der ganzen App

React + Vite + TypeScript · läuft **ohne Backend** · alle Flakiness-Quellen sind bewusst eingebaut

Denkmodell: Given / When / Then

Jeder Test erzählt eine kleine Geschichte:

```
test('Bestellung zeigt Erfolgs-Toast', async ({ page }) => {  
  // Given – Ausgangszustand  
  await page.goto('/')  
  
  // When – Aktion  
  await page.getByRole('searchbox').fill('Mojito')  
  await page.getByRole('button', { name: 'In den Warenkorb' }).click()  
  await page.getByRole('button', { name: 'Bestellen' }).click()  
  
  // Then – erwartetes Ergebnis  
  await expect(page.getByText('Prost! 🍷')).toBeVisible()  
})
```

GWT (fachlich) $\approx$ **AAA** – *Arrange / Act / Assert* (technisch). Gleiche Idee, andere Brille.

Der Code-Generator

Playwright schreibt den ersten Test für dich mit:

```
npm run dev # App auf http://localhost:5173  
npx playwright codegen http://localhost:5173
```

- Klick dich durch die App → Playwright generiert den Test-Code live mit
- Wählt automatisch **rollenbasierte Locators** (`getByRole` , `getByLabel`)
- Perfekt für den **Einstieg** und zum Locator-Lernen
- **Aber:** Assertions, Logik & Zeit-Themen muss man selbst ergänzen

VSCoDe-Integration

Die Extension **Playwright Test for VSCoDe**:

- Tests **direkt aus dem Editor** starten & debuggen (grüner Pfeil)
- **Pick Locator** — Element im Browser anklicken, Locator wird eingefügt
- **Record new** — Codegen direkt im Editor
- Breakpoints & Step-Debugging im Testcode

Demo · Übung 1.1 mit codegen

Wir machen den **Anfang gemeinsam** — danach übernehmt ihr.

```
npm run dev # App auf http://localhost:5173  
npx playwright codegen http://localhost:5173
```

Übung 1.1 — Caipirinha konfigurieren, Dialog-Überschrift & Preis 9,00 € prüfen: aufzeichnen, Assertion ergänzen, in die Spec-Datei übernehmen, laufen lassen.

📁 Workshop-Repo · Datei: e2e/1-codegen.spec.ts · Aufgaben: uebungen/1-codegen.md

Übung 1 · Code-Generator

Aufgabe — weiter mit 1.2 bis 1.4

- **1.2** Konfigurator mit Checkbox, Radio, Zahlenfeld, Notiz
- **1.3** Fünf gleiche „In den Warenkorb“-Buttons — welcher ist gemeint?
- **1.4** Menge im Warenkorb erhöhen und die neue Anzahl prüfen

Lernziel

Wann hilft Codegen — und wann muss ich selbst ran?

📁 Workshop-Repo · Datei: `e2e/1-codegen.spec.ts` · Aufgaben: `uebungen/1-codegen.md`

Recap & Fragen · Einführung & Codegen



2 · Locators, Actions & Assertions

Teststruktur: describe, test, beforeEach

```
import { test, expect } from '@playwright/test'

test.describe('Bestellprozess', () => {
  test.beforeEach(async ({ page }) => {
    await page.goto('/') // Arrange: gleicher Startpunkt
  })

  test('einen Cocktail bestellen', async ({ page }) => {
    // ...
  })

  test('mehrere Cocktails bestellen', async ({ page }) => {
    // ...
  })
})
```

Jeder `test` bekommt eine **frische** `page` → saubere Isolation, kein geteilter State.

Locators – die Rangfolge

Von **robust & nutzerorientiert** (oben) zu **brüchig** (unten):

1. `getByRole('button', { name: 'Bestellen' })` – Rolle + Zugänglichkeit
2. `getByLabel('Auf welchen Namen?')` – Formularfelder
3. `getByPlaceholder('Cocktail suchen...')`
4. `getByText('Prost!')`
5. `getByTestId('cart-drawer')` – wenn nichts anderes greift
6. CSS / XPath – nur als letztes Mittel

`getByRole` hilft dabei **besseres, barriereärmeres Markup** zu schreiben.

Locator Good Practices am Beispiel

```
const suche = page.getByRole('searchbox', { name: 'Cocktail suchen' })
const name = page.getByLabel('Auf welchen Namen?')
const bestellen = page.getByRole('button', { name: 'Bestellen' })
```

- **Locator ist lazy** — wird bei jeder Nutzung neu aufgelöst (kein `StaleElement`)
- An **Semantik** gekoppelt, nicht an DOM-Struktur → überlebt Refactorings
- Lesbar wie Fachsprache — auch für manuelle Tester
- Sieht die Seite wie **assistive Technologien** (Screenreader) → Barrierefreiheits-Lücken fallen beim Testschreiben schon auf

Actions

```
await suche.fill('Mojito') // Textfeld setzen
await suche.pressSequentially('Mojito') // Zeichen-für-Zeichen (Key-Handler)
await page.getByRole('checkbox', { name: 'alkoholfrei' }).check()
await page.getByRole('radio', { name: 'Rum' }).check()
await page.getByRole('button', { name: 'Bestellen' }).click()
```

- `fill` vs. `pressSequentially` — je nachdem, ob die App auf einzelne Tasten reagiert
- Vor **jeder** Action prüft Playwright automatisch die Actionability

Assertions – Web-First & Auto-Retry

```
// wartet automatisch, bis sichtbar (bis zum Timeout)
await expect(page.getByText('Prost! 🍺')).toBeVisible()

// wartet, bis Button aktivierbar
await expect(page.getByRole('button', { name: 'Bestellen' })).toBeEnabled()

// wartet auf konkreten Text
await expect(page.getByTestId('gesamtpreis')).toHaveText('12,50 €')
```

Web-First Assertions haben eingebautes **Auto-Retry** – kein `sleep`, kein manuelles Pollen.

Test-Isolation — „gratis“ durch Browser Contexts

- Jeder Test läuft in einem eigenen **Browser Context** — wie ein frisches Inkognito-Fenster
- In **Millisekunden** erzeugt, kein Browser-Neustart
- Frischer State: keine geteilten Cookies, kein `localStorage`
- Parallelisierung & Isolation **out of the box** → weniger Cross-Test-Flakiness

Gegenpol zum **Shared-State**-Problem — dazu später mehr bei Flakiness.

UI-Mode — Grundlagen

```
npx playwright test --ui
```

- **Watch-Mode:** Tests laufen bei jeder Änderung neu
- **Zeitleiste** jeder Aktion mit DOM-Snapshot
- **Locator-Picker** direkt eingebaut
- Netzwerk, Konsole, Source — alles in einem Fenster

Der beste Startpunkt zum interaktiven Entwickeln & Debuggen von Tests.

Demo · Übung 1.5 vervollständigen

Bevor wir zu Locators & Assertions gehen: den **letzten Test aus Übung 1** gemeinsam zu Ende bringen.

- Aufgezeichneten Filter-Workflow noch mal durchgehen
- Fehlende **Assertions** ergänzen (Anzahl der Suchergebnisse)
- Locators aus dem Codegen kritisch prüfen — was bleibt, was wird ersetzt?

📁 Workshop-Repo · Datei: `e2e/1-codegen.spec.ts`

Demo · Übung 2.1 im UI-Mode

Wir schreiben **2.1 gemeinsam** — dann übernehmt ihr.

```
npx playwright test e2e/2-locators-assertions.spec.ts --ui
```

Übung 2.1 — Startzustand prüfen: Überschrift „Happy Hour 🍹“ (`getByRole('heading')`), Zähler „Bestellungen heute: 0“ (`getByText`), genau **fünf** Cocktail-Karten (`getByRole('article')`).

Dabei live: Pick Locator, Watch-Mode.

📁 Workshop-Repo · Datei: `e2e/2-locators-assertions.spec.ts` · Aufgaben: `uebungen/2-locators-assertions.md`

Übung 2 · Locators & Assertions

Aufgabe — weiter mit 2.2 bis 2.5

- **2.2** Mojito in den Warenkorb — Preis scopen, `data-testid` als Fallback
- **2.3** Suche & Filter grenzen die Liste ein
- **2.4** Warenkorb: Menge erhöhen, Position entfernen, Drawer schließen
- **2.5** Konfigurator, Validierungsfehler, erfolgreiche Bestellung

Lernziel

Sauberer Test mit semantischen Locators & Web-First-Assertions.

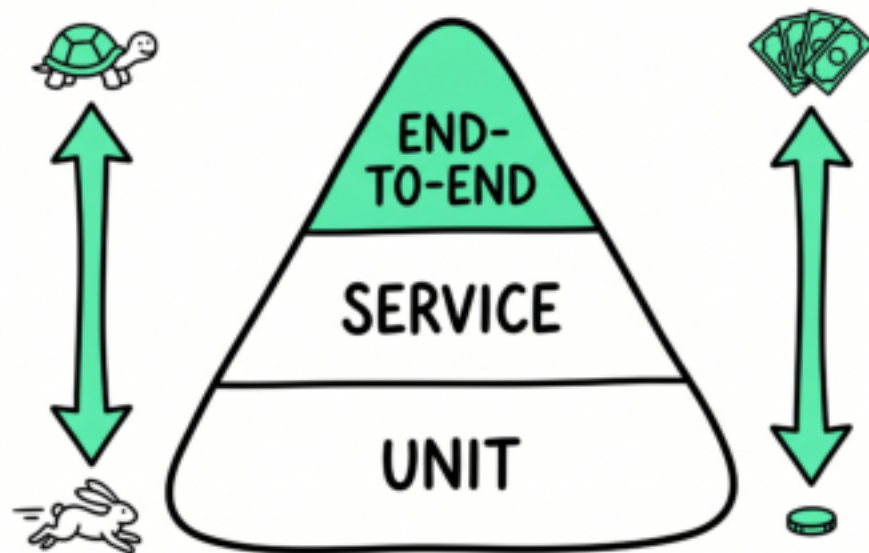
📁 Workshop-Repo · Datei: `e2e/2-locators-assertions.spec.ts` · Aufgaben: `uebungen/2-locators-assertions.md`

Recap & Fragen · Locators, Actions & Assertions



3 · Testing Good Practices

Die Testpyramide



Ziel: **Breite Testabdeckung** in einer akzeptablen Laufzeit.

Worum es in der Pyramide wirklich geht

Nicht um die **Namen der Ebenen** — „Unit“, „Service“, „Integration“ sind bei jedem anders geschnitten.

Die Erkenntnis ist simpler:

Viele schnelle Tests, wenige langsame. Und: **viele billig zu wartende, wenige teure.**

Playwright verschiebt das Verhältnis ein Stück nach oben — es *ist* schnell. Aber ein Unit-Test bleibt um Größenordnungen schneller als ein E2E-Test. Die Regel bleibt.

...aber die Basis allein reicht nicht



Jedes Teil funktioniert **für sich** — nur **zusammen** funktionieren sie nicht.

Mocks lügen: der Nachbar verhält sich in echt anders

Wann überhaupt einen E2E-Test?

E2E ja, wenn...

- kritischer **User-Flow** (z.B. Bestellung end-to-end)
- Zusammenspiel vieler Teile
- echtes Risiko bei Bruch

Ziel: **Breite Testabdeckung in einer akzeptablen Laufzeit.**

Warnung: das Ice-Cream-Cone-Anti-Pattern

Die Gefahr nach einem Workshop wie diesem: **alles** wird jetzt E2E getestet.

Pyramide ▲

- breite Basis schneller Tests
- Spitze aus wenigen E2E-Tests
- Suite läuft in Minuten

Ice Cream Cone 🍦

- oben eine dicke Schicht E2E
- kaum etwas darunter
- Suite läuft in Stunden – und flackert

E2E-Tests sind **wichtig** – aber nie das **Einzig**e im Portfolio.

Welche Testarten es genau braucht, hängt an eurer **Architektur- und Teststrategie** – das sprengt diesen Workshop.

Lesbarkeit — weniger ist mehr

- Tests an der **Anwendung** orientieren, nicht an Tickets
- **AAA-Struktur** sichtbar machen (Arrange / Act / Assert)
- **Sprechende Namen aus der Domäne** statt technischer Kürzel
- **DAMP** statt **DRY** — Lesbarkeit schlägt Redundanzfreiheit ([Google Testing Blog](#))

Faustregel: Ein Test sollte sich **vorlesen** lassen.

Page-Object-Pattern

Wiederkehrende Interaktionen bündeln:

```
class CocktailBarPage {  
  constructor(private page: Page) {}  
  
  async filter(name: string) {  
    await this.page.getByRole('searchbox').fill(name)  
  }  
  async inDenWarenkorb(name: string) {  
    await this.page.getByRole('button', { name: `${name} in den Warenkorb` }).click()  
  }  
  async bestellen(gast: string) {  
    await this.page.getByLabel('Auf welchen Namen?').fill(gast)  
    await this.page.getByRole('button', { name: 'Bestellen' }).click()  
  }  
}
```

Der Test liest sich dann wie Fachsprache: `bar.filter('Mojito')`, `bar.bestellen('Anna')`.

Umgang mit Timeouts

- **Nicht** globale Timeouts hochdrehen, um Flakiness zu „lösen“
- Auf **Zustände** warten (`expect( ... ).toBeVisible()`), nicht auf **Zeit**
- Kein `page.waitForTimeout(3000)` – das ist `Thread.sleep` im neuen Gewand
- Auto-Retry nutzen: `await expect(locator).toHaveCount(2)`
- Falls nötig den Timeout einer langsamen Aktion gezielt anpassen:

```
await expect(locator).toHaveText('hello', { timeout: 10_000 });
```

```
// ❌ so nicht  
await page.waitForTimeout(3000)  
// ✅ so  
await expect(page.getByTestId('gesamtpreis')).toHaveText('12,50 €')
```

Demo · Übung 3.1 — hartes Warten

Sieben Tests, jeder mit **genau einem** Problem. Sechs sind grün und trotzdem falsch.

Übung 3.1 machen wir gemeinsam:

1. `waitForTimeout(2000)` löschen → Test wird **rot** („Expected: 1, Received: 5“)
2. Reparieren **ohne** neues Warten: `expect(await ...count())` → `await expect(locator).toHaveCount(1)`

Die App ist in Ordnung — schuld ist immer der Test. `src/` bleibt unangetastet.

📁 Workshop-Repo · Datei: `e2e/3-good-practices.spec.ts` · Aufgaben: `uebungen/3-good-practices.md` · Lösung: `3-good-practices.solution.spec.ts`

Übung 3 · Good Practices

Aufgabe — weiter mit 3.2 bis 3.8

Pro Test: **Problem benennen**, dann reparieren.

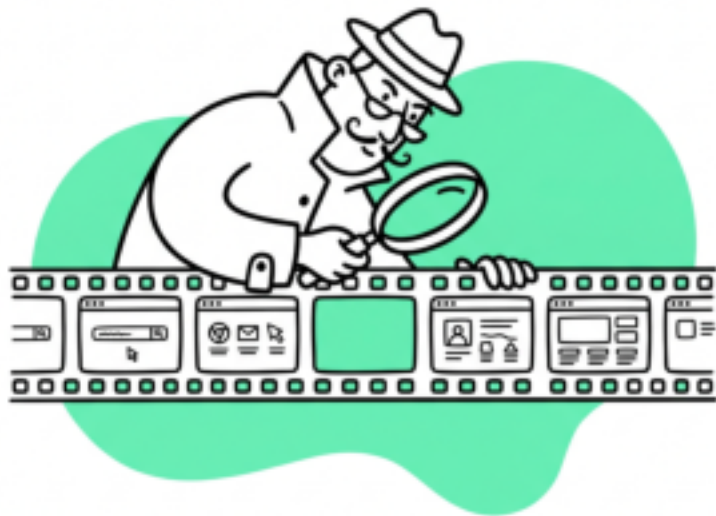
- XPath & CSS statt semantischer Locators
- Assertion ohne `await`
- Query statt Web-First-Assertion
- **3.8** kombiniert alle Probleme in einem längeren Test

Lernziel

Wartbarkeit & Verlässlichkeit.

📁 Workshop-Repo · Datei: `e2e/3-good-practices.spec.ts` · Aufgaben: `uebungen/3-good-practices.md`

Recap & Fragen · Good Practices



4 · Debugging

Ein Test schlägt fehl – jetzt was?

Playwright gibt dir vier Werkzeuge an die Hand:

1. **HTML-Report** – was ist wo & warum durchgefallen
2. **Screenshot** – Zustand zum Fehlerzeitpunkt
3. **Video** – der ganze Ablauf
4. **Trace Viewer** – die Zeitreise durch den Test

Alles **automatisch** bei Fehlern – nichts selbst zusammenbauen.

HTML-Report

```
npx playwright test          # erzeugt den Report  
npx playwright show-report    # öffnet ihn im Browser
```

- Übersicht aller Tests: grün / rot / geflaked
- Pro Test: Schritte, Fehlermeldung, Artefakte
- Direkter Einstieg in Screenshot, Video & Trace

Screenshot & Video

In `playwright.config.ts`:

```
export default defineConfig({  
  use: {  
    screenshot: 'only-on-failure',  
    video: 'retain-on-failure',  
    trace: 'on-first-retry',  
  },  
})
```

- **Screenshot** — schneller Blick auf den Endzustand
- **Video** — zeigt den Weg dahin (z.B. Sidebar-Animation)
- Nur bei Fehler gespeichert → keine unnötigen Artefakte

Trace Viewer – die Zeitreise

```
npx playwright test --trace on  
npx playwright show-trace trace.zip
```

- **DOM-Snapshot** zu **jedem** Schritt – vor/während/nach der Aktion
- **Netzwerk, Konsole, Source & Timings** synchron
- Fehlersuche **ohne Rätselraten**: „Was war da wirklich auf der Seite?“

Ideal für Actionability-Probleme: „Warum war der Button noch nicht klickbar?“

Demo · Übung 4.1 im Trace-Viewer

Fünf rote Tests, fünf **verschiedene** Fehlerbilder. Ziel ist nicht so sehr der Fix, sondern das **Verstehen der Ursache**.

```
npx playwright test e2e/4-debugging.spec.ts  
npx playwright show-report           # roter Test → Trace-Viewer
```

Übung 4.1 gemeinsam: Menge von **Cuba Libre** erhöhen — die Assertion schlägt fehl. Am Vorher/Nachher-Snapshot sehen wir: der Klick hat die **falsche Zeile** getroffen.

📁 Workshop-Repo · Datei: `e2e/4-debugging.spec.ts` · Aufgaben: `uebungen/4-debugging.md`

Übung 4 · Trace-Viewer

Aufgabe — weiter mit 4.2 bis 4.5

Jeder Test braucht ein **anderes** Werkzeug im Trace-Viewer:

- **4.2** Element im DOM, aber nicht sichtbar → DOM-Inspector
- **4.3** fehlgeschlagener API-Call → Netzwerk-Tab
- **4.4** stiller JavaScript-Fehler → Konsole
- **4.5** Race Condition beim Doppelklick → Zeitleiste

📁 Workshop-Repo · Datei: `e2e/4-debugging.spec.ts` · Aufgaben: `uebungen/4-debugging.md`

Lernziel

Fehlersuche ohne Raten — **Actionability** & Zeitreise verstehen.

Recap & Fragen · Debugging



5 · Flakiness

Was ist Flakiness?

- Ein Test, der **mal grün, mal rot** ist — **ohne** dass sich der Code ändert
- Der teuerste Testtyp: er **kostet Vertrauen** ins ganze Test-Set
- „Läuft bei mir“ / „einfach nochmal starten“ → schleichender Verfall

Flakiness ist meist ein **Timing-** oder **State-Problem**.

Die vier Flakiness-Szenarien

Alle vier stecken bewusst in der Cocktail-Bar.

1 · Langsamer Server

Suche mit künstlichem Delay – Test prüft zu früh.

2 · Shared State zwischen Tests

Warenkorb leckt in den nächsten Test.

3 · Verzögerte Hydration

Klick vor Ende der Hydration ist ein No-op.

4 · Zeitabhängiges Verhalten

Happy-Hour-Preis hängt an der echten Uhrzeit.

Flakiness-Szenario 1 · Langsamer Server

Problem: Die Suche filtert mit künstlichem Delay – der Test prüft zu früh.

```
// ❌ flaky  
await page.getByRole('searchbox').fill('Mojito')  
await page.waitForTimeout(500)  
expect(await page.getByRole('listitem').count()).toBe(1) // zu früh!
```

```
// ✅ Web-First-Assertion retryt bis zum Ergebnis  
await page.getByRole('searchbox').fill('Mojito')  
await expect(page.getByRole('listitem')).toHaveCount(1) // wartet automatisch
```

`await ... count()` ist ein Momentwert. `expect(locator).toHaveCount()` **wartet**.

Flakiness-Szenario 2 · Shared State zwischen Tests

Problem: Drei Tests teilen sich eine `page` – der Warenkorb wächst mit.

```
// ❌ geteilte page, serieller Modus, State leckt zwischen Tests
test.describe.configure({ mode: 'serial' })
let page: Page
test.beforeAll(async ({ browser }) => { page = await browser.newPage() })

// ✅ page-Fixture pro Test – frischer Browser-Context
test.beforeEach(async ({ page }) => {
  await page.goto('/')
})
```

Jeder Test bekommt einen eigenen Context → kein Storage-, kein React-, kein Warenkorb-Leak.

Flakiness-Szenario 3 · Verzögerte Hydration

Problem: Button ist sichtbar & nicht disabled, Handler aber noch nicht registriert. Ein fester `waitForTimeout` trifft mal die Hydration, mal nicht — Klick wird zum No-op.

```
// ❌ flaky — Auto-Wait meldet grün, Handler ist trotzdem noch nicht da
await page.waitForTimeout(300)
await page.getByRole('button', { name: 'Bestellen' }).click()
```

```
// ✅ auf den erwarteten Effekt warten — Klick notfalls wiederholen
await expect(async () => {
  await page.getByRole('button', { name: 'Bestellen' }).click()
  await expect(page.getByRole('dialog', { name: 'Bestellung' })).toBeVisible()
}).toPass()
```

Playwrights Actionability sieht die Hydration nicht — auf ein direktes Klick-Ergebnis warten.

Flakiness-Szenario 4 · Zeitabhängiges Verhalten

Problem: Happy-Hour-Rabatt hängt an der echten Uhrzeit → abends grün, morgens rot.

```
// ❌ Test hängt an der Wanduhr des CI-Servers
await expect(page.getByTestId('preis')).toHaveText('6,80 €')

// ✅ Zeit deterministisch setzen – mitten in die Happy Hour
await page.clock.setFixedTime(new Date('2026-07-11T17:30:00'))
await page.goto('/')
await expect(page.getByTestId('preis')).toHaveText('6,80 €')
```

`page.clock` friert die Zeit ein → **deterministisch**, unabhängig von Zeitzone & Tageszeit. `setFixedTime` **vor** `goto` – sonst sieht die App die reale Zeit beim Laden.

Werkzeugkasten gegen Flakiness

Timing

- Auto-Wait und Auto-Retry nutzen
- Web-First-Assertions (`expect(locator)`)
- `waitForTimeout` nur in begründeten

Ausnahmefällen

State

- Browser-Context-Isolation
- `page.clock` für Zeit
- Netzwerk mocken

Regel: **Warte auf Zustände, nicht auf Zeit. Kontrolliere, was der Test nicht besitzt.**

Demo · Übung 5.1 — fehlender Wait

```
npx playwright test e2e/5-flaky-tests.spec.ts  
npx playwright test e2e/5-flaky-tests.spec.ts --repeat-each=5 # Flakiness sichtbar machen
```

Übung 5.1 gemeinsam: Der Test tippt „mojito“, wartet stur 500 ms und prüft dann auf eine Karte — der Filter greift aber erst nach einer zufälligen Verzögerung von 0–1000 ms. Mal trifft der Wait, mal nicht → mal grün, mal rot.

Fix: Query → **Web-First-Assertion**, die bis zum Ergebnis retryt.

📁 Workshop-Repo · Datei: `e2e/5-flaky-tests.spec.ts` · Aufgaben: `uebungen/5-flaky-tests.md` · Lösung: `5-flaky-tests.solution.spec.ts`

Übung 5 · Flakiness stabilisieren

Aufgabe — weiter mit 5.2 bis 5.4

Lernziel

Flakiness-Ursachen **benennen** & gezielt beseitigen.

- **5.2** geteilter Zustand zwischen drei Tests
(`beforeAll` + `serial`)
- **5.3** feste Wartezeit vor dem Bestellen (verzögerte Hydration)
- **5.4** Happy-Hour-Preis hängt an der echten Uhrzeit
→ `page.clock`

Zuerst die **Ursache benennen**, dann fixen — nicht bloß grün bekommen.

📁 Workshop-Repo · Datei: `e2e/5-flaky-tests.spec.ts` · Aufgaben: `uebungen/5-flaky-tests.md`

Recap & Fragen · Flakiness



6 · Optionale Themen

Der einzige `fetch`: Coupon-Check

Die ganze App ist clientseitig — **außer** dem Gutschein-Feld:

```
POST /api/validate-coupon → { valid: bool, rabattProzent: number }
```

- Idealer, **isolierter** Aufhänger fürs Mocking
- Hat einen **lokalen Fallback** → Demo bricht nie ab
- Damit lassen sich Fehlerfälle testen, die man sonst nie zu Gesicht bekommt

page.route() – Antworten mocken

```
// gültiger Coupon
await page.route('**/api/validate-coupon', route =>
  route.fulfill({
    json: { valid: true, rabattProzent: 20 },
  }),
)

// Fehlerfall: Server-Fehler
await page.route('**/api/validate-coupon', route =>
  route.fulfill({ status: 500 }),
)
```

Netzwerk **entkoppelt** → stabile, reproduzierbare Testdaten & echte Fehlerpfade.

Demo · Coupon mocken

Freie Übung — hier gibt es **keine** vorbereitete Spec-Datei. Wir legen sie gemeinsam an:

```
touch e2e/6-mocking.spec.ts
```

Gemeinsam: den **gültigen** Coupon mocken (`valid: true, rabattProzent: 20`) und die sichtbare Preisänderung prüfen. Die Fehlerfälle baut ihr dann selbst.

📁 Workshop-Repo · neue Datei: `e2e/6-mocking.spec.ts` · Der einzige `fetch` der App: `POST /api/validate-coupon`

Zusatz-Übung · Request-Mocking

Aufgabe — weiter in eurer eigenen Datei

- **ungültige** Antwort setzen → Fehlermeldung prüfen
- Server-Fehler simulieren (**500**)
- **langsame** Antwort simulieren (`route` + Delay)
- sichtbare **Preisänderung** bzw. **Fehlermeldung** prüfen

Lernziel

Netzwerk entkoppeln, stabile Testdaten, Fehlerpfade testen.

📁 Workshop-Repo · Datei: `e2e/6-mocking.spec.ts` (selbst anlegen)

Recap & Fragen · Request-Mocking

Testdaten über API – test-only Endpoint

Eure Fachfrage: Daten aus Playwright heraus per Backend anlegen.

- Eigener **test-only Endpoint** – nur **außerhalb Produktion** verfügbar
- Aus Playwright über den `request` **-Context** ansprechen (nicht über die UI)
- In **Fixtures** kapseln: anlegen → Test → **Cleanup** (Isolation!)
- Auth & Daten-Reset mitdenken

```
const res = await request.post('/test-api/seed', { data: { cocktails: 3 } })
```



Live-Demo Component-Tests

AI-Integration

Playwright + AI — passend zu eurer Infra:

- **Codegen** als Ausgangspunkt, **AI verfeinert** Assertions & Struktur
- **Test-Healer**: kaputte Locators automatisch reparieren lassen
- Eigene Modelle in eurer Infra: **Qwen Coder 3.6**, **opencode**, **Claude Code**
- Playwright **MCP-Server** — Agenten steuern den Browser direkt

Realistisch bleiben: AI beschleunigt, ersetzt aber kein Testverständnis.

Danke! 🍹

Fragen, Feedback & offene Punkte

Viel Erfolg beim Umstieg von Selenium auf Playwright.